CHAPTER I

1. What is Amdahl's law? Explain it with required equation and suitable example.

→ Amdahl's law : It is used to find the maximum/ overall improvement possible by improving a particular part of a system.

Amdahl's law uses two factors to find speedup from some enhancements.

① Fraction enhanced. (f')
② Speed up enhanced. (s')

Proof:

If

s = overall speed up
T = old execution time
T' = new execution time
t = execution time of a fraction to be improved.
t' = new execution time of a fraction ~~to~~ after improvement.
tn = execution time that is taken by a portion that won't be enhanced.
f' = fraction enhanced.
s' = speed up enhanced.

$$s = \frac{T}{T'}$$

$$T = t_n + t$$

$$T' = t_n + t'$$

$$f' = t/T$$

$$= \frac{t}{t + t_n}$$

$$1 - f' = 1 - \frac{t}{t + t_n}$$

$$1 - f' = \frac{t + t_n - t}{t + t_n}$$

$$1 - f' = \frac{t_n}{t + t_n} \quad -(a)$$

$$s' = \frac{t}{t'}$$

$$t' = t/s'$$

$$= \frac{T \times f'}{s'}$$

$$= \frac{(t_n + t) f'}{s'}$$

$$\therefore \frac{t'}{t_n + t} = \frac{f'}{s'} \quad -(b)$$

$$s = \frac{T}{T'} = \frac{t_n + t}{t_n + t'} = \frac{1}{\left[\frac{t_n + t'}{t_n + t}\right]}$$

$$= \frac{1}{\frac{t_n}{t_n + t} + \frac{t'}{t_n + t}}$$

$$= \frac{1}{1 - f' + \frac{f'}{s'}}$$

[$\because$ from (a) & (b)]

e.g. problem of Amdahl's law:

Find f.

$$\text{Soverall} = 4 = \frac{1}{1 - f + \frac{f}{16}}$$

$$\Rightarrow 4 = \frac{1}{\frac{16 - f + f}{16}}$$

$$\Rightarrow 64 - 60f = 16$$

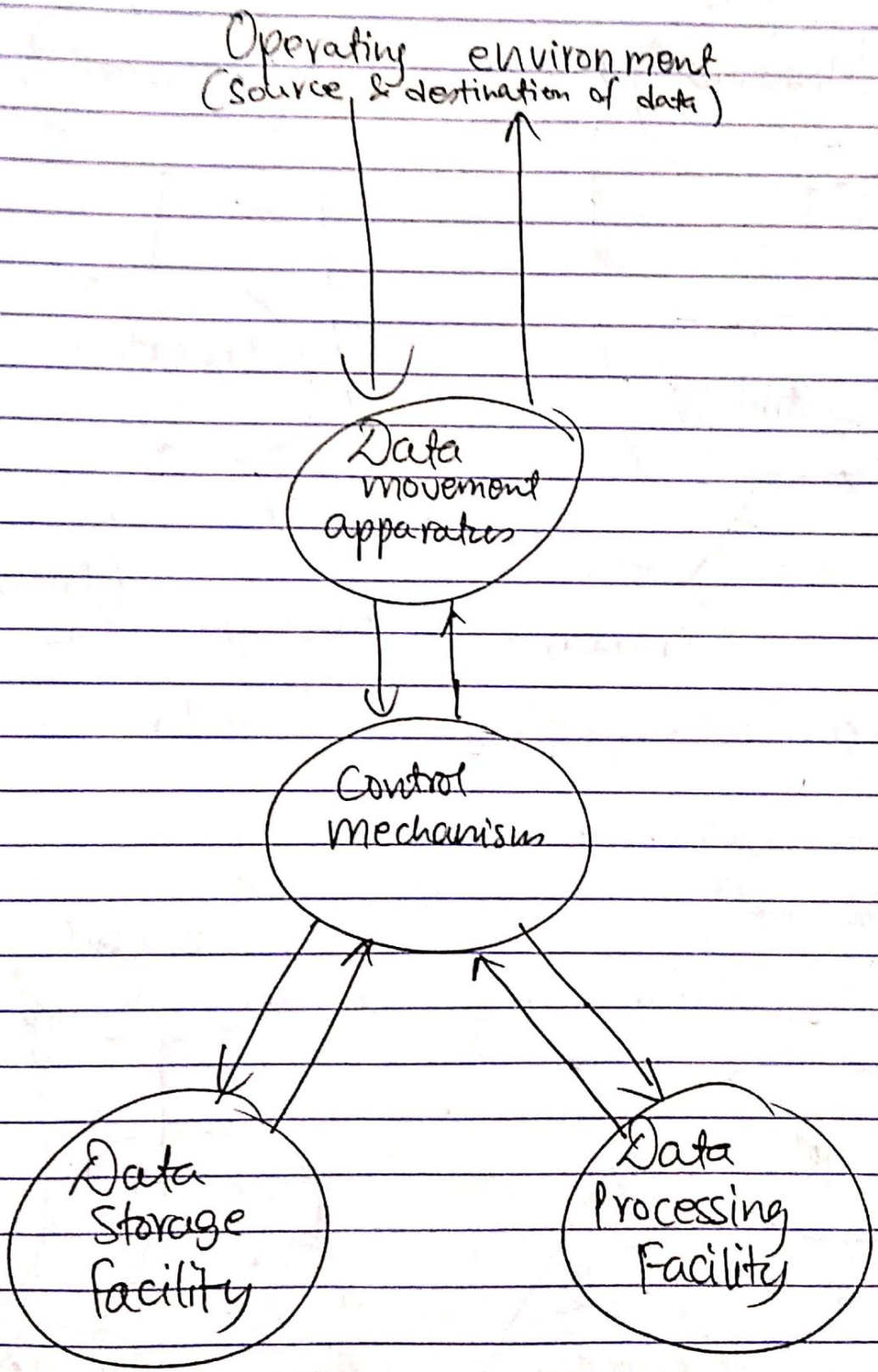
$$\therefore f = 80\%$$

$\therefore$ 80% of vectorization needed.

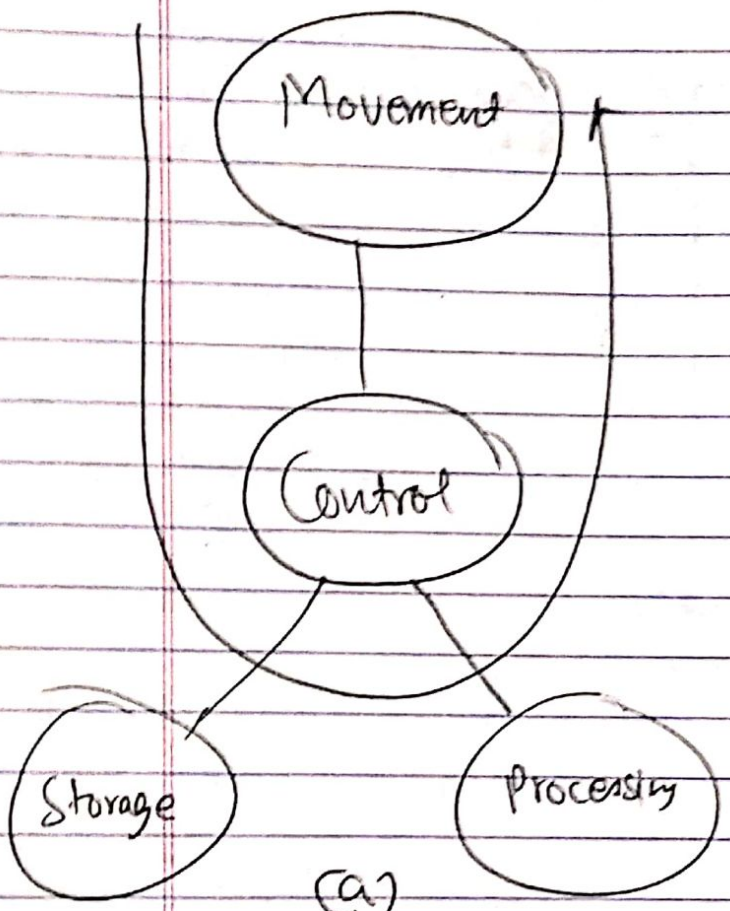
Q. no. 2.

Computer Organization	Computer Architecture
① Describes how the computer perform	① Describe what the computer does.
② Implementation	② Design
③ Control signals, circuits are predominant.	③ This is much related to programming like instruction set, addressing mode.
④ Computer organization changes as per changes in technology.	④ Computer architecture (like Harvard, V.N) e lasts for longer time actually way longer time
⑤ the way ~~computer~~ hardware components operate and the way they are connected together to form computer system.	⑤ Concerned with the structure and behavior as seen by user.

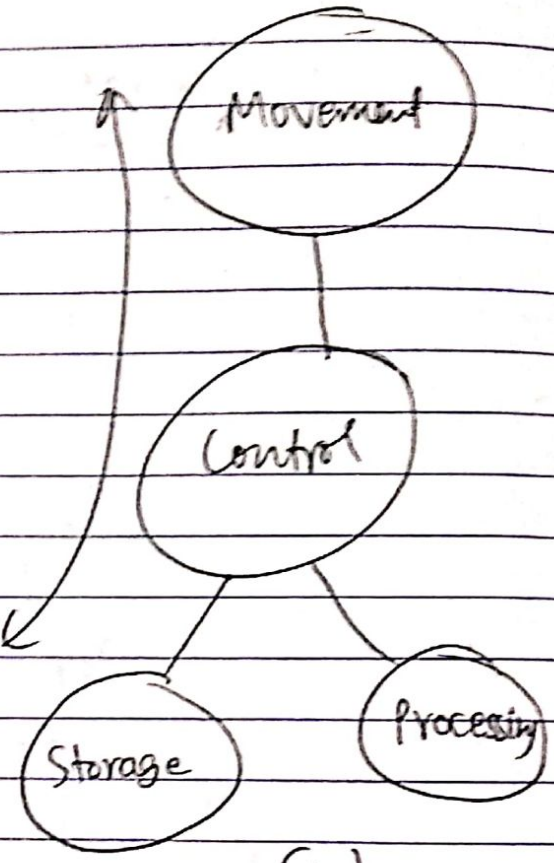
3. Functional view of computer with suitable example.



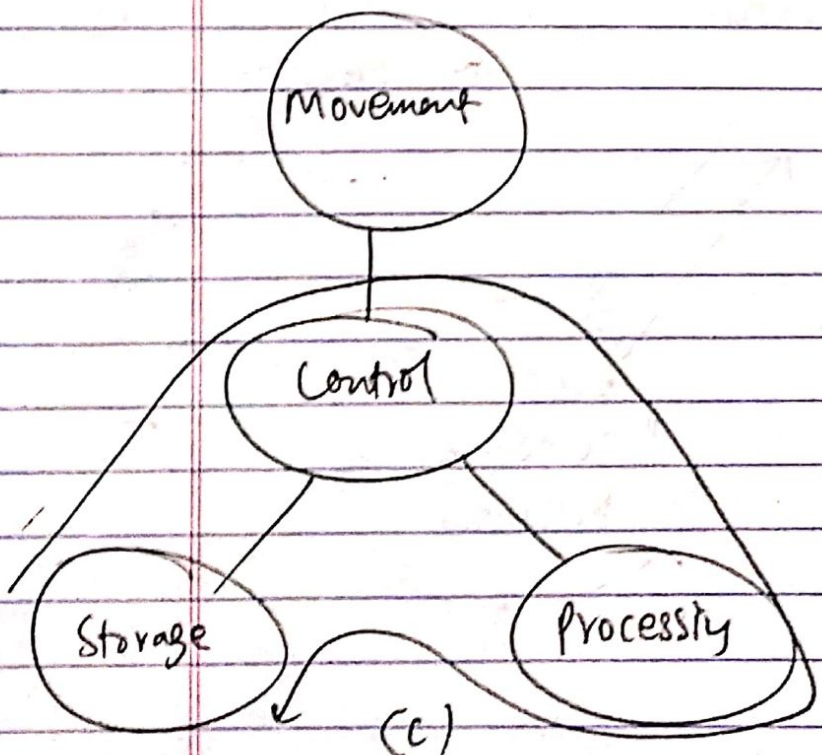
Operations



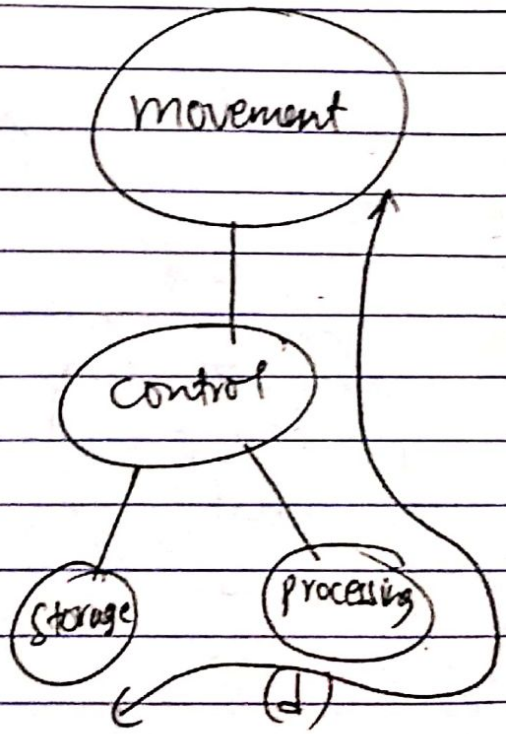
(a)



(b)



(c)



(d)

Figure(a): The computer can work as data movement device.

Figure (b): It can function as data storage device.

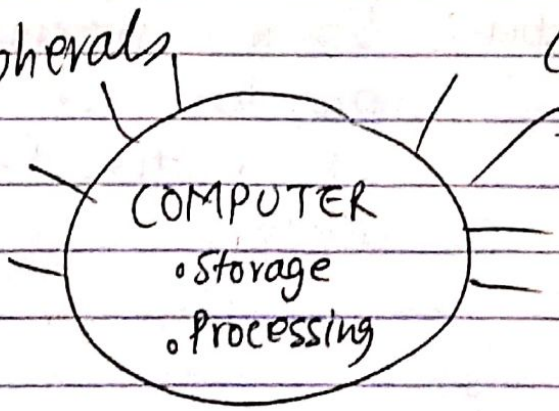
Figure (c) ~~(d)~~: Operations involving data processing, on data ~~either~~ in storage

Figure (d): Operations involving data processing on data en-route between storage and the external environment.

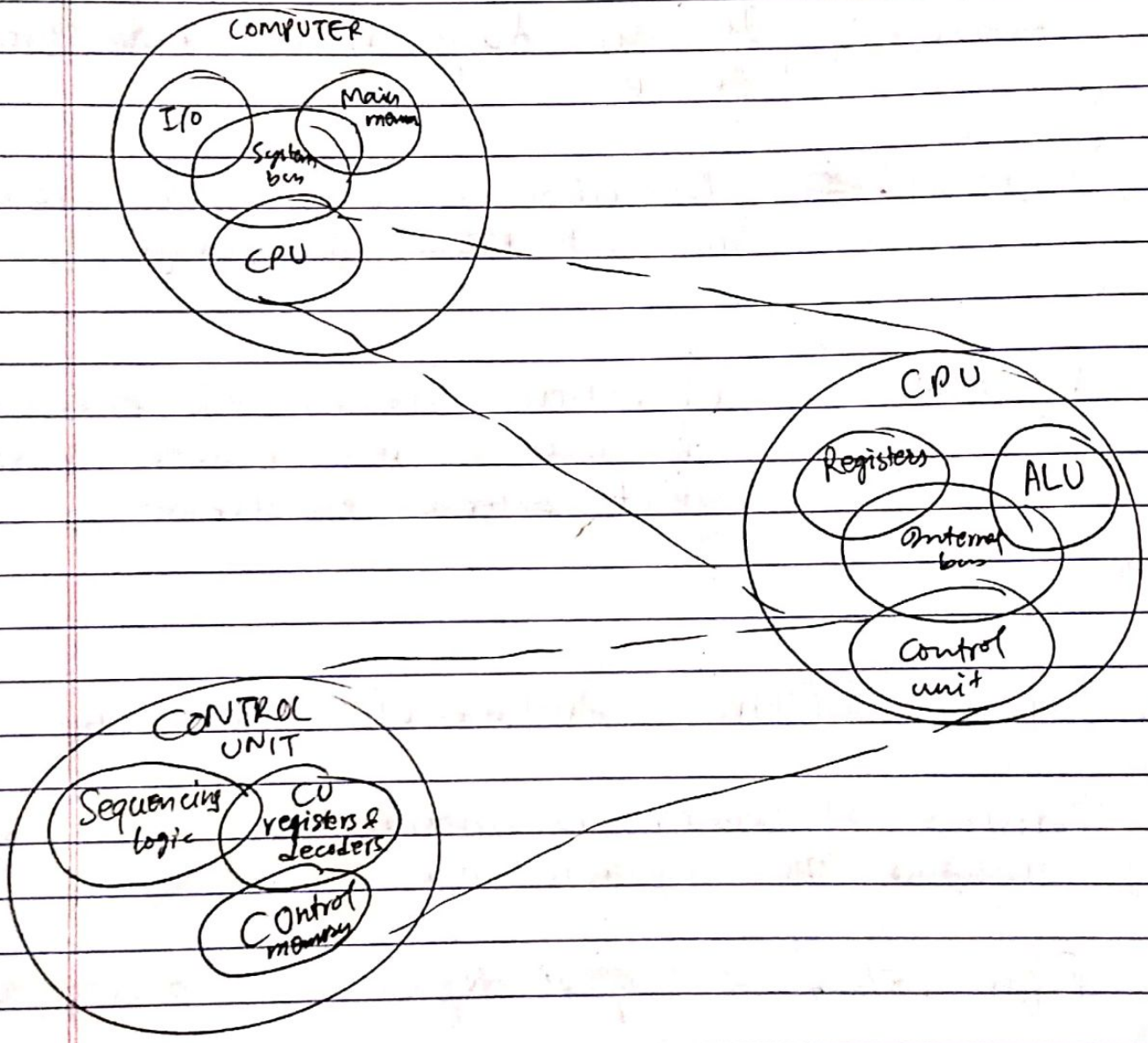
4. Explain different structures of a computer.

Structure of computer is concerned with the way in which the components are inter-related.

Figure below is simplest depiction of a computer.

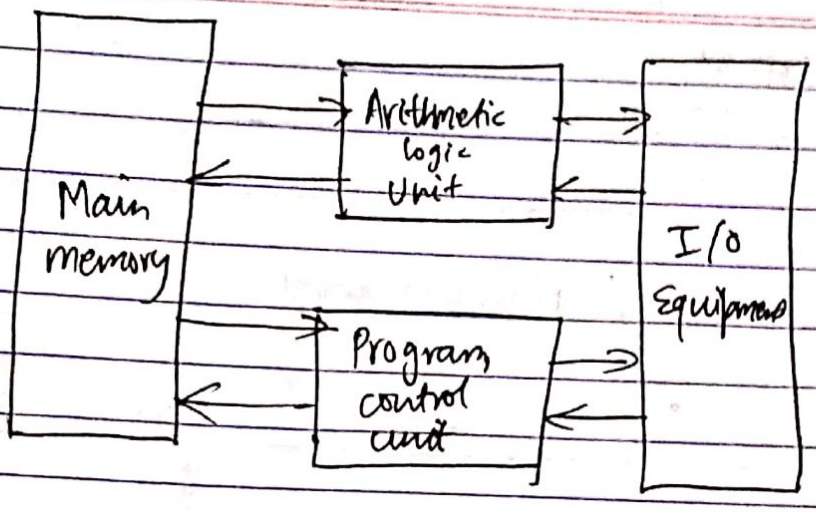


This diagram can be further expanded as follows:



CPU interconnection: Some mechanism that provides for communication among the CU, ALU and registers.

8. no. 5.



In 1946, V.N. pioneered IAS computer.

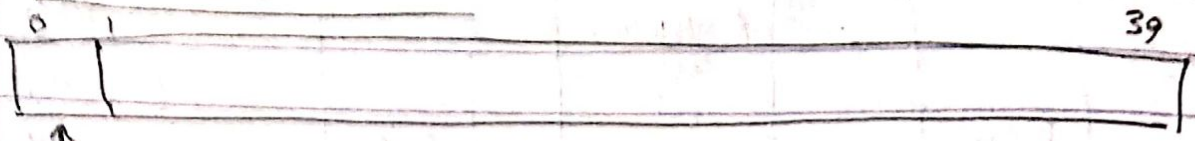
Structure of the IAS Computer

The memory of the IAS consists of 1000 storage locations, called words, of 40 binary digits (bits) each.

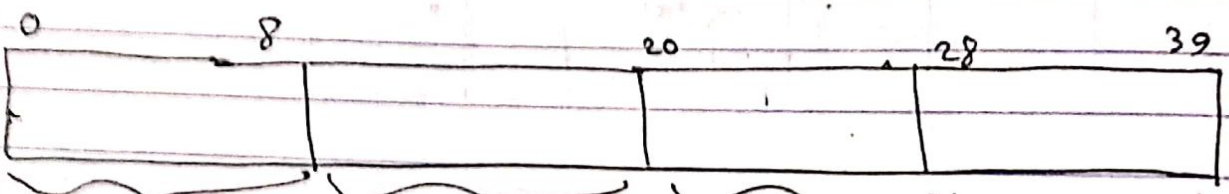
Both data and instructions are stored there. Numbers are represented in binary form and each instruction is a binary code.

main memory: Stores both data & instructions.
ALU: capable of operating on binary data.
CU: interprets the instructions in memory and causes them to be executed.
I/O: equipment operated by the control unit Main memory (M).

IAS memory Formats:



(a) Number word.



(b) Instruction word.

Figure: 2.2 IAS Memory Format.

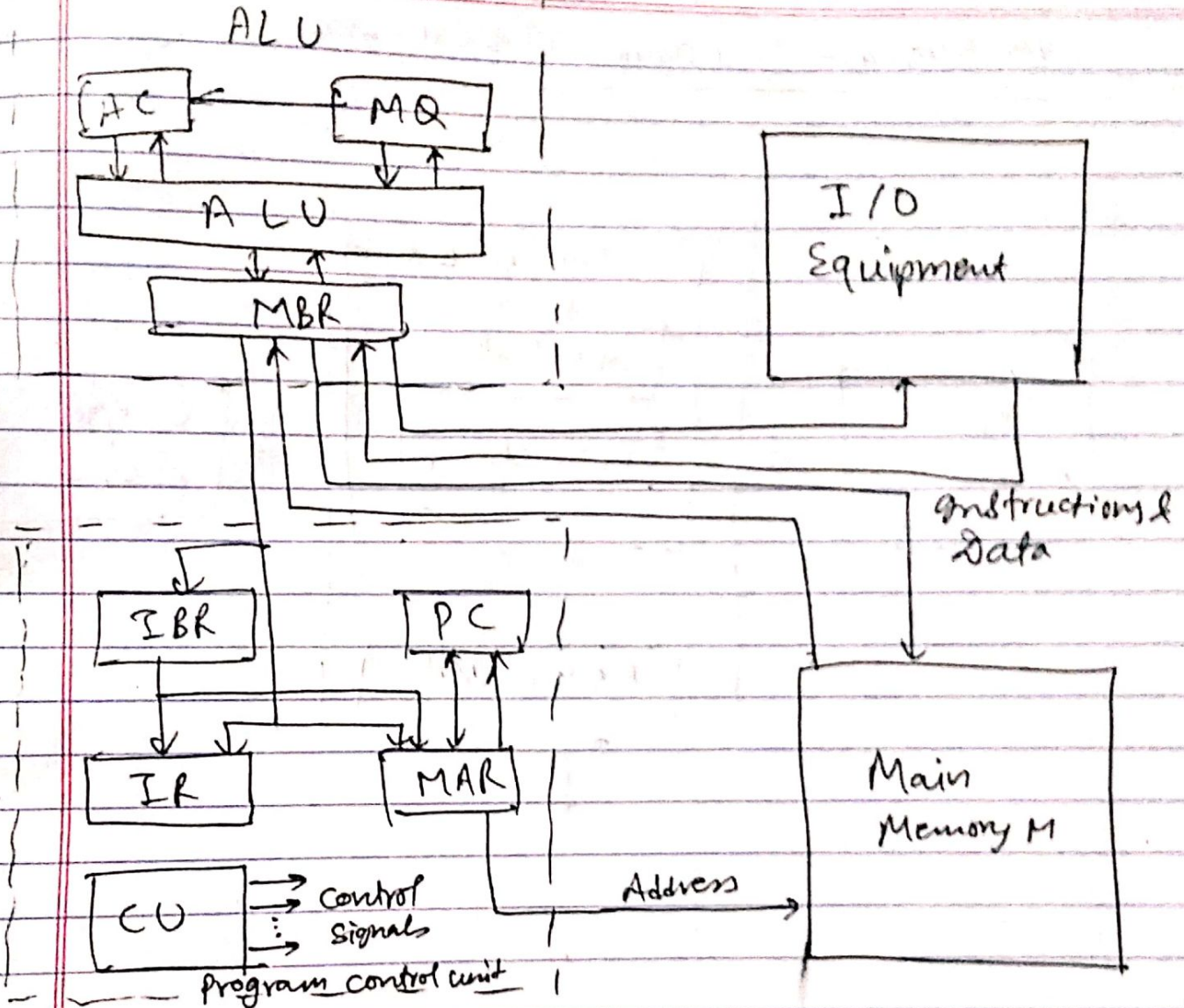
MBR: Contains a word to be stored in memory or sent to the I/O unit, or is used to receive a word from memory or I/O unit.

IR: Contains the 8-bit opcode instruction being executed

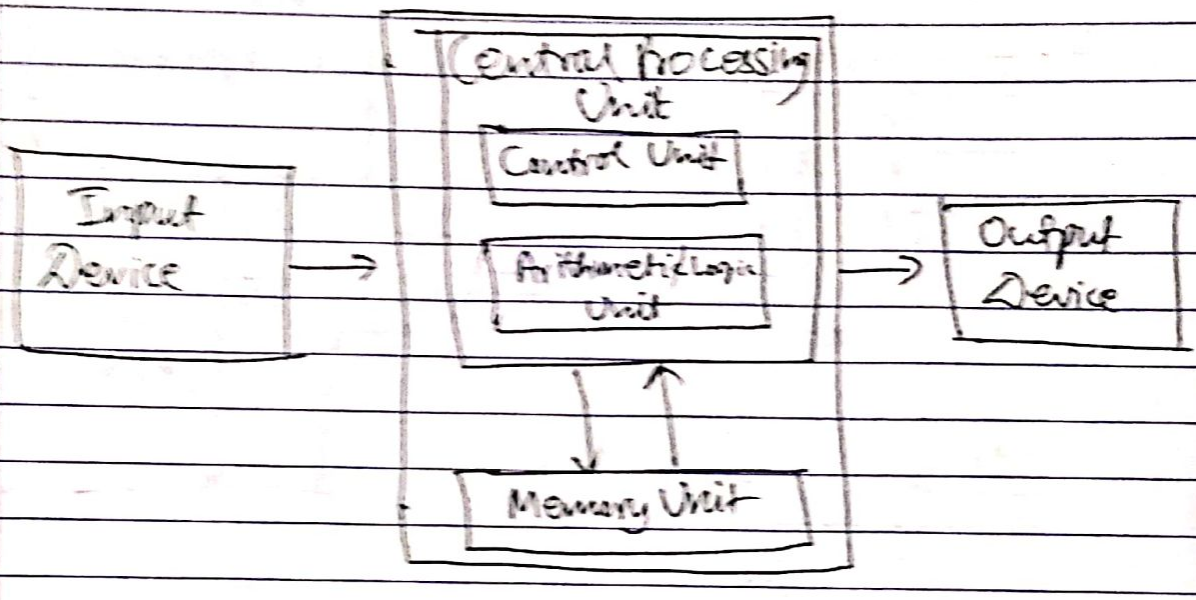
IBR: Employed to hold temporarily the right-hand instruction from a word in memory

PC: contains the address of the next instruction pair to be fetched from memory.

AC/MQ: Accumulator & Multiplier Quotient are employed to hold temporary operands and results of ALU operations. For e.g. the result of multiplying two 40-bit numbers is an 80-bit number; the most significant 40 bits are stored in the AC and the least significant in MQ.


ALU
AC
MQ
ALU
MBR
I/O
Equipment
Instructions &
Data
IBR
PC
IR
MAR
Main
Memory M
CU
Control
Signals
Address
Program control unit

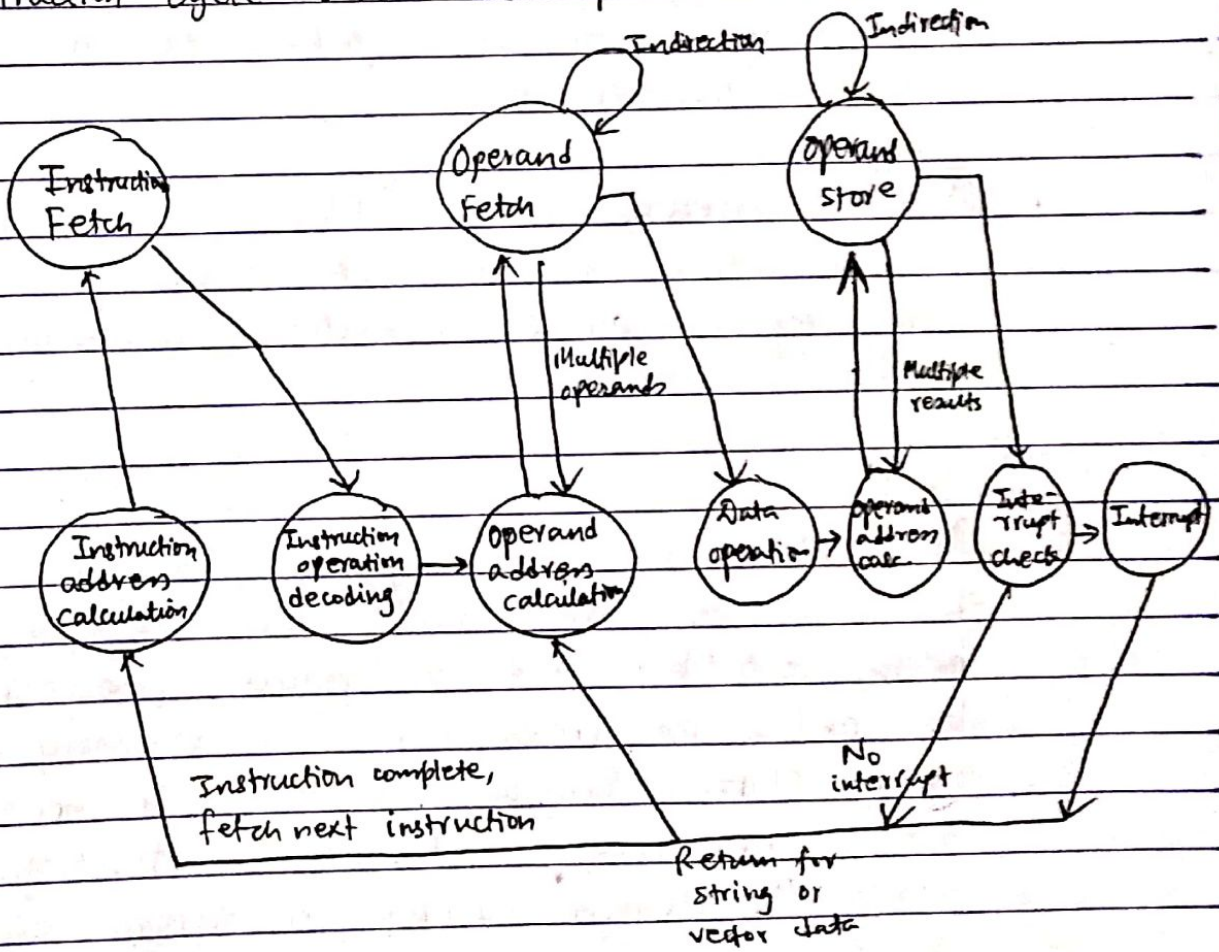
6. Von Neumann machine block diagram



1 Why state Diagram is required? Explain the state diagram of instruction cycle with interrupts.

A state diagram is used to represent the condition of the system or part of the system at finite instances of time.

It's a behavioral diagram and it represents the behavior using finite state transitions. State diagrams can be used to represend instruction cycle and instruction cycle with interrupts also.



Each phase of instruction cycle can be decomposed into a sequence of elementary micro-operations.

~~Instruction~~

The cycles are basically,

- Fetch cycle
- Indirect cycle
- Execute cycle
- Interrupt cycle.

1. The Fetch cycle:-

At the beggining of the fetch cycle, the address of the next instruction to be executed is in Program counter (PC).

StepI:

The address in the PC is moved to the memory address register (MAR), as this is the only register which is connected to address lines of the system bus.

StepII

The address in MAR is placed on the address bus, now the control unit issues a READ command on the control bus, and the result appears on the data bus and is then copied into the memory buffer register (MBR). Program counter is incremented by one, to get ready for the next instruction. (These two actions can be performed simultaneously to save time.)

Step III:
The content of the MBR is moved to the instruction Register. (IR)

Thus, a simple Fetch cycle consist of three steps and four micro-operations. Symbolically, we can write these sequence of events as follows:

t_1: MAR ⟵ PC

t_2: MBR ⟵ MEMORY

PC ⟵ (PC) + I

t_3: IR ⟵ (MBR)

To accomodate interrupts, an interrupt cycle is added to the instruction cycle. In the interrupt cycle, the process checks if any interrupts, indicated by the presence of an interrupt signal.

If no interrupts are pending, the processor proceeds to the fetch cycle and fetches the next instruction of the current program. If an interrupt is pending, the processor does the following:

- Suspends execution of the current program being executed and saves its content. This means saving the address of the next instruction to be executed (current contents of

the program counter) and any other data relevant to the processor's current activity

- It sets the program counter to the starting address of an interrupt handler routine.

The processor now proceeds to the fetch cycle and fetches the first instruction in the interrupt handler program, which will service the interrupt.

The interrupt handler program is generally part of the operating system. Typically, this program determines the nature of the interrupt and performs whatever action are needed.

In the example we have been using, the handler determines which I/O module generated the interrupt and may branch to a program that will write more data out to that I/O module.

When the interrupt handler routine is completed, the processor can resume execution of the user program at the point of interruption.

RISC	CISC
→ RISC processors have simple instructions taking about one clock cycle. The average clock cycle per instruction (CPI) is 1.5	→ CISC processor has complex instructions that take up multiple clocks for execution. The average clock cycle per instruction (CPI) is in the range 2 and 15.
→ The length of instructions is fixed in RISC architecture.	→ The length of instructions is variable in CISC architecture.
→ As a result, RISC architecture is highly pipelined.	→ CISC architecture is not highly pipelined.
→ So code execution time is also less.	→ In CISC, because of variable and usually longer instruction length, code execution time is more.
→ used in high-end applications such as video processing, telecommunications and image processing.	→ CISC architecture is used in low-end applications such as security systems, home automation, etc.

3. Different types of operations used in a computer:

- Data Transfer
- Arithmetic
- Logical
- Conversion
- I/O
- System Control
- Transfer of Control

Data Transfer

→ Specifies location of source and destination operands
  ↳ location could be memory, a register, or the top of the stack.
  ↳ length of the data to be transferred should be indicated.
  ↳ with all instructions, with operations the mode of addressing for each operand must be specified.

e.g.

L → load → 32 bits → Transfer from memory to register
LH → Load Halfword → 16 bits → Transfer from memory to register
LR → load → 32 bits → Transfer from register to register.

Arithmetic : Some examples of single operand instructions are:

- Absolute — Take the absolute value of the operand.
- Negate — Negate the operand.
- Increment — Add 1 to the operand
- Decrement — Subtract 1 from the operand.

(iii) Logical.

P	Q	$\bar{P}$	P.Q	P+Q	P(XOR)Q
0	0	1	0	0	0
0	1	1	0	1	1
1	0	0	0	1	1
1	1	0	1	1	0

Suppose X = 101110

Y = 111010

$X \oplus Y$ = 010100

Logical shift: Logical shift right means the bit on leftmost will remain same while other bits are moved towards right one step ahead. The lsb is lost.

e.g. 1101 —shift right→ 1110

logical shift left means the bit on rightmost will remain same while other bits are moved one step ahead towards left. The msb is lost.

e.g. 1011 —shift left→ 1101

Arithmetic shift: Its same like logical shift but the bit is emptied (instead of being copied) on other side of shifted direction.

for e.g. if shifted right, the leftmost bit is empty.
if shifted left, the rightmost bit is empty.

Rotate: or cyclic shift, operations preserve all of the bits being operated on.

(iv) Conversion.
→ change of format; for example, binary to decimal.

(v) Input/Output.
For input/output, there are a variety of approaches taken including isolated programmed I/O, memory mapped programmed I/O, DMA, and the use of an I/O processor.

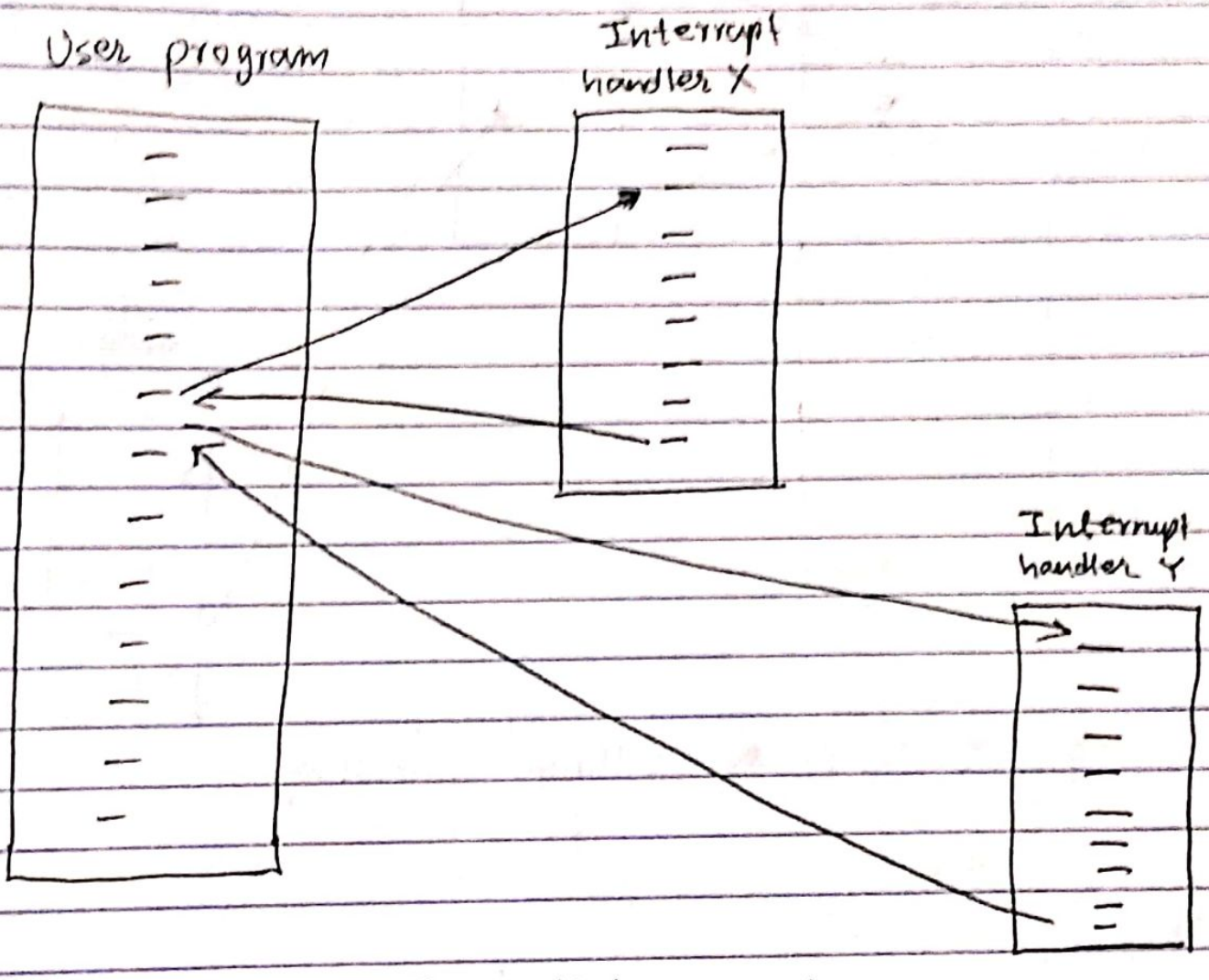
(vi) System Control
→ can be executed only while the processor is in a certain privileged state or is executing a program in a special privileged area of memory.

(vii) Transfer of Control
e.g.s.
Jump (branch) – Unconditional Transfer; load PC with specified addres or do nothing, based on condition.

Jump to Subroutine – Return – Execute – Skip.
Halt – wait – Skip Conditional

~~Zero, One, Two~~

1. Explain the state diagram of instruction cycle (with interrupts). How can you handle multiple interrupts?



(a) Sequential interrupt processing

Two approaches can be taken to dealing with multiple interrupts.

① Disabled interrupts — ignore interrupt signal.

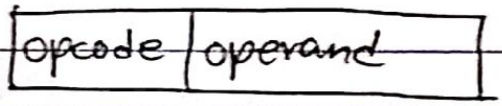
② Define priorities for interrupts and to allow an interrupt of higher priority to cause a lower-priority interrupt handler to be itself interrupted.

⑤ Addressing in computer

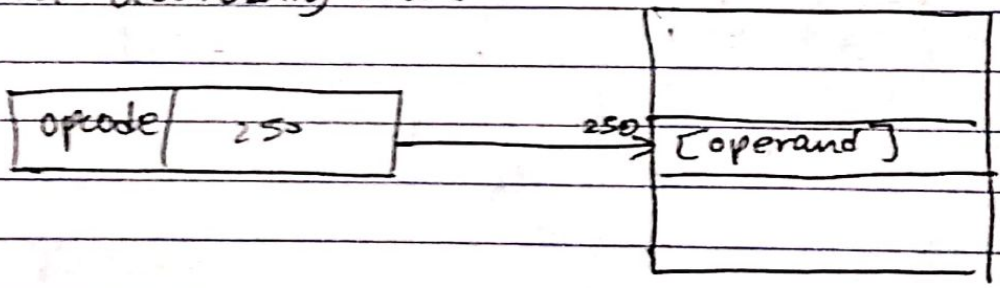
In a computer, addres is a location of data, usually in main memory or on a disk. We can think of computer memory as an array or storage boxes, each of which is one byte in length. Each box has an addres (a unique number) assigned to it. By specifying a memory addres (i.e. by addressing), programmers can access a particular byte of data.

Addressing modes:

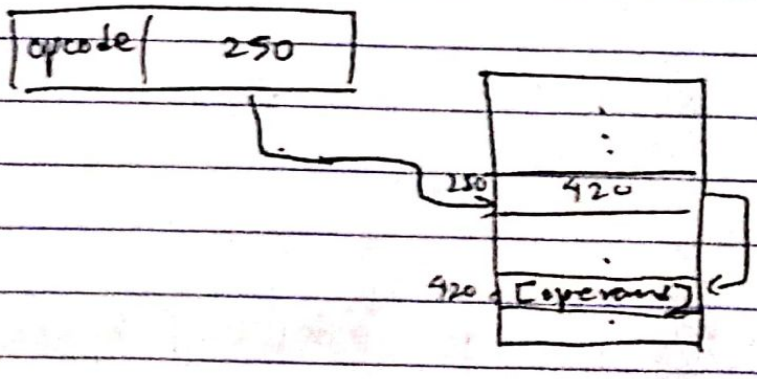
① Immediate addressing mode



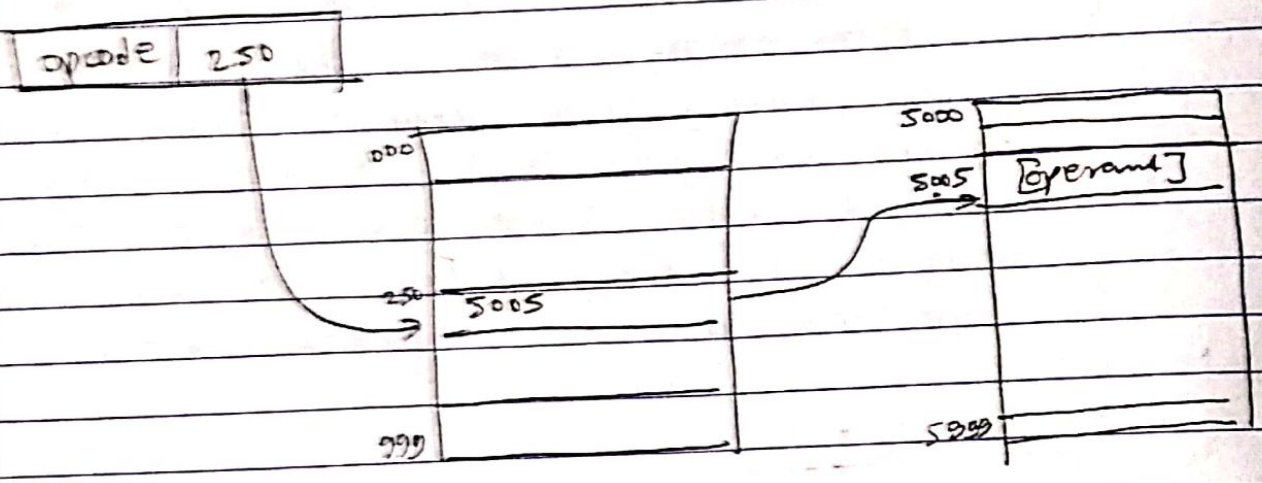
② Direct addressing mode



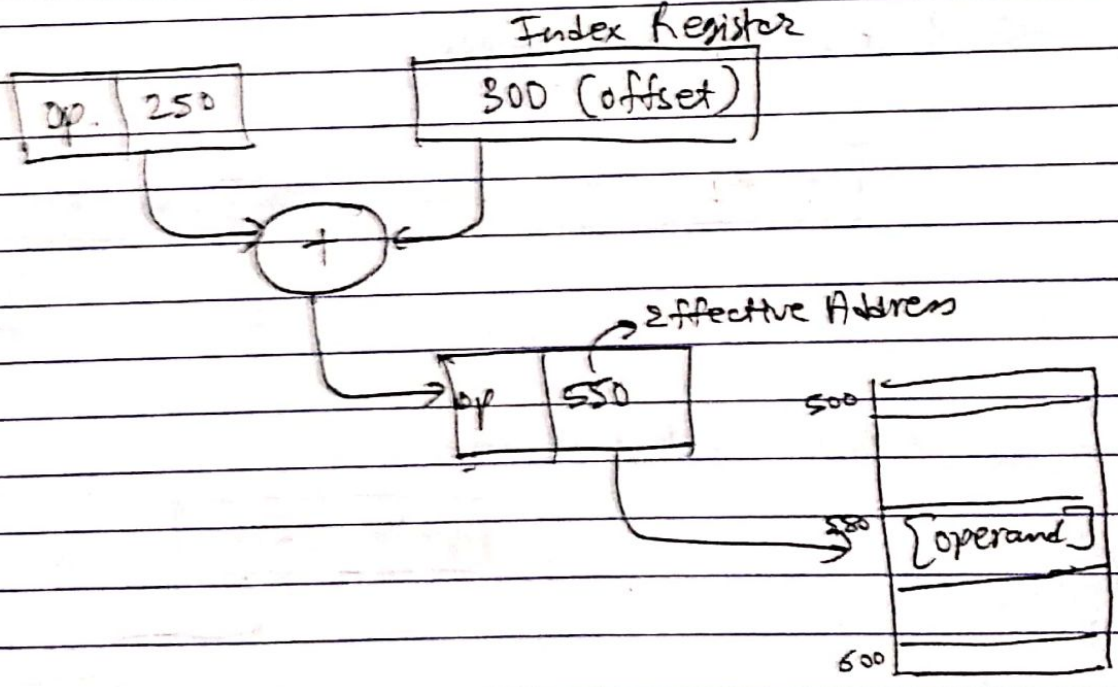
③ Indirect addressing mode



(d) Register direct addressing mode.



(e) Index Addressing mode



(f) Base Addressing mode: Same as Index Addressing mode. Only the difference is, it has Base register instead of index register.

CHAPTER III

1. Consider a i386 system and it has 40 address lines. Calculate the total memory it can address.

Design a logic circuit to address three 1024 MB RAMS and two 512 MB RAMS.

Soln

Since there are 40 address bus, it can address, 2^{40} = 1.09 trillion addresses.

Second problem:

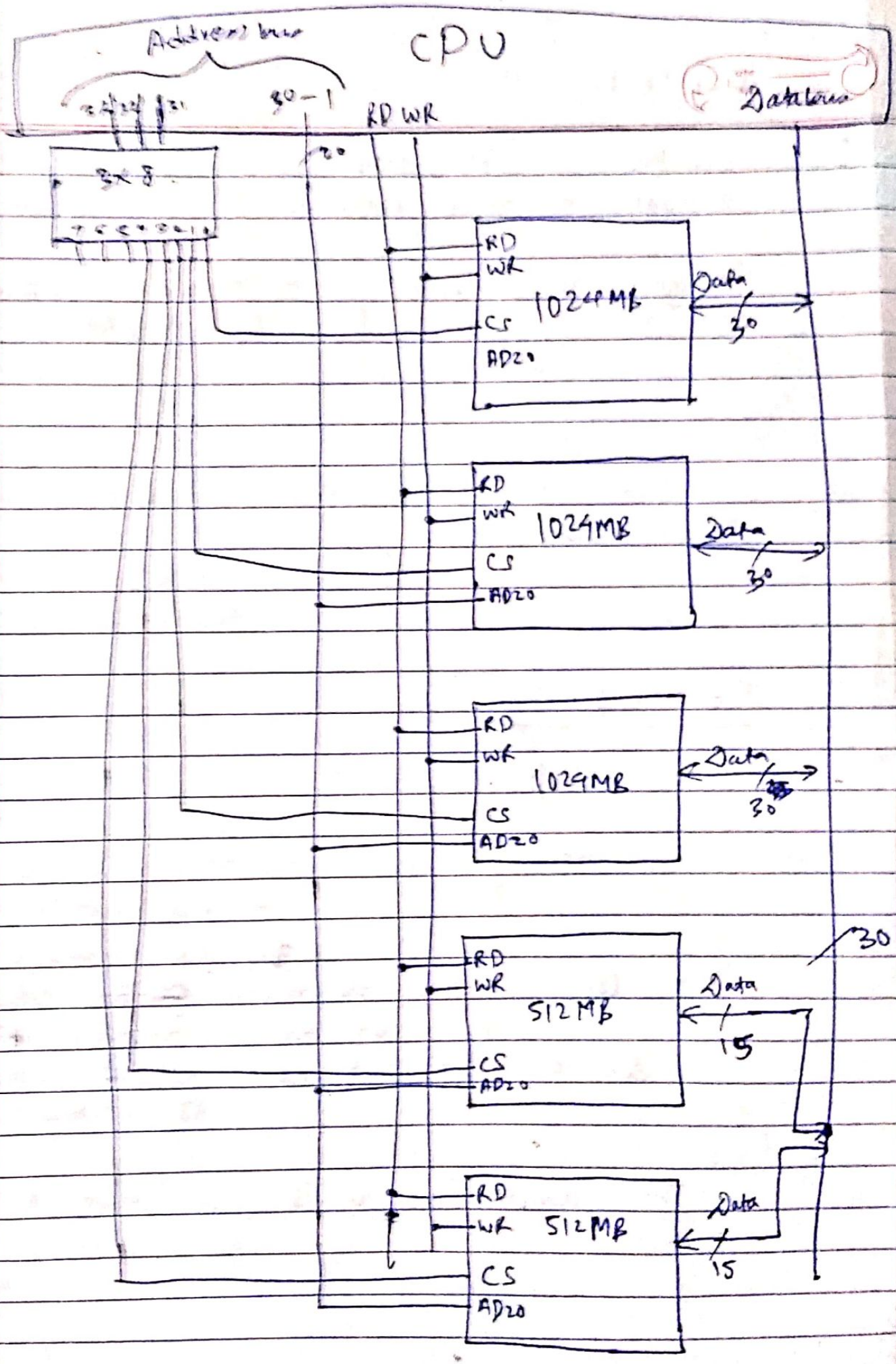
No. of chips 3+2=5.

No. of memory addresses = 1024x1024 in each module. (1024x)

i.e. it require ~~$2^{10} = 1024$~~, 30 address bus. ($\because 2^{20} = 1024 \times 1024$)

The Chip selection ~~should be 15~~ 5. will require 5 lines. So we need a total of 15 output in decoder. So let's use ~~4x16 Decoder~~ 3x8 Decoder.

The address bus willbe at least 5+20=25 bits.


CPU
Address bus
31
30-1
RD WR
Data bus
3x8
20
RD
WR
CS
AD20
1024MB
Data
30
1024MB
1024MB
512MB
512MB
15
30

2. Why RAID System? Explain its principles, standard levels and implementation in detail.

RAID (Redundant Array of Independent Disks) is a data storage virtualization technology that combines multiple physical disk drive components into a single logical unit for the purpose of data redundancy, performance improvement, or both.

Data is distributed across the drives in one of several ways, referred to as RAID levels, depending on the required level of redundancy and performance.

Each schema, or a RAID level, provides a different balance among the key goals:

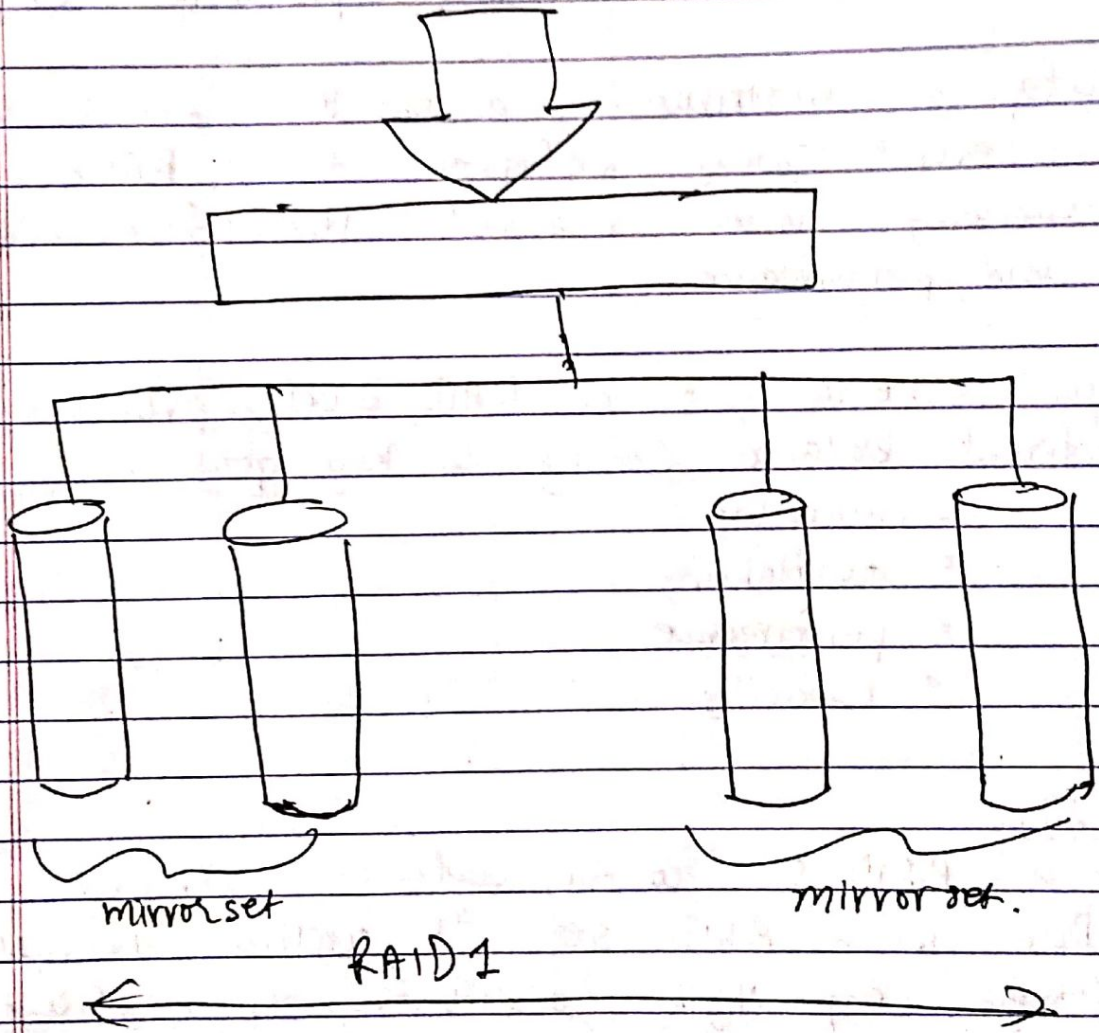
- reliability
- availability
- performance
- capacity.

RAID 0.

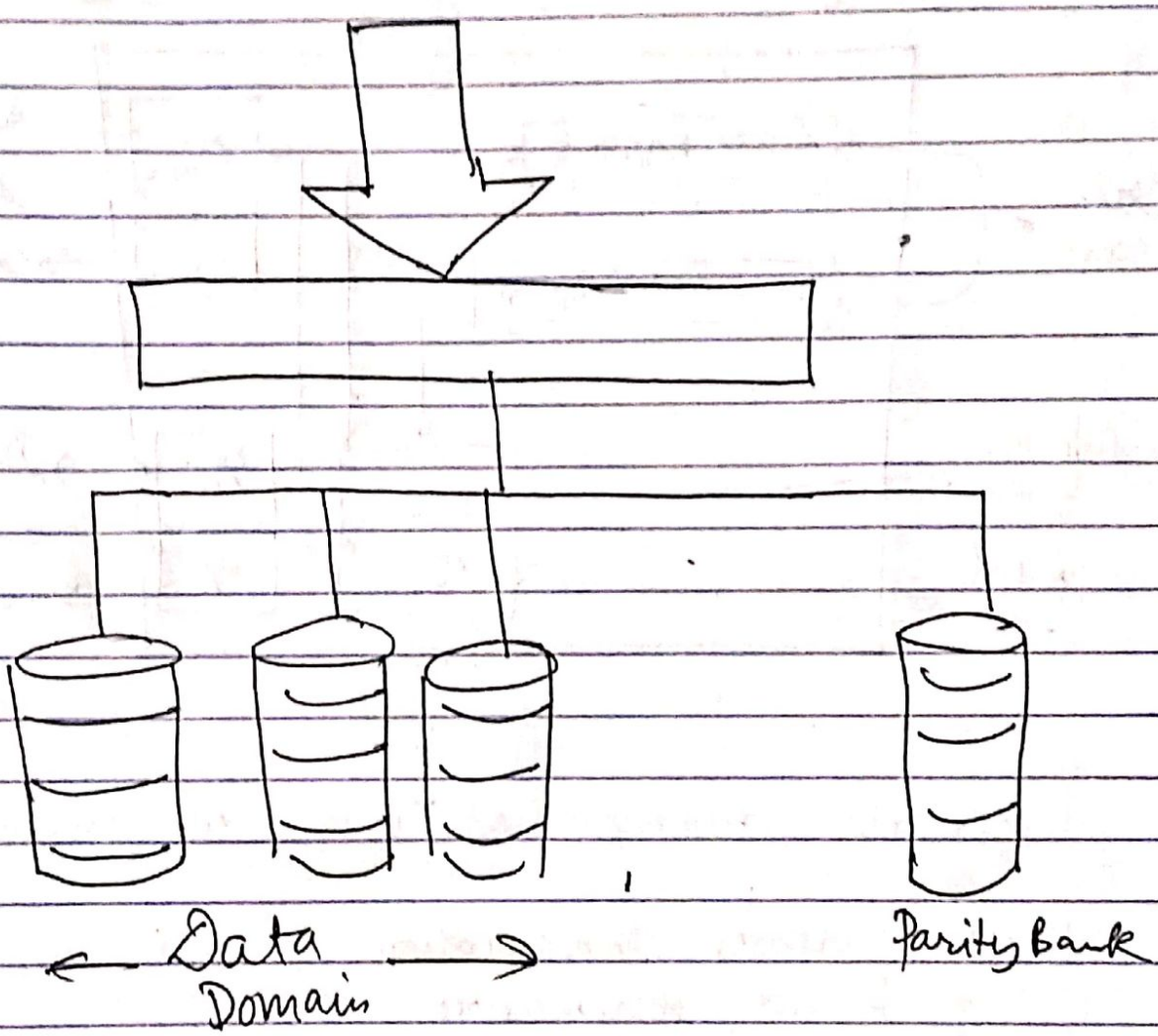
- In a RAID 0 config, data is striped across the HDDs in a RAID set. It utilizes the full storage capacity by distributing strips of data over multiple HDDs in a RAID set.
- To read data, all the strips are put back together by the controller.

RAID 1: data is mirrored to improve fault tolerance. A RAID 1 group consists of at least two HDDs.

→ As explained in mirroring, every write is written to both disks, which is transparent to the host in a hardware RAID.

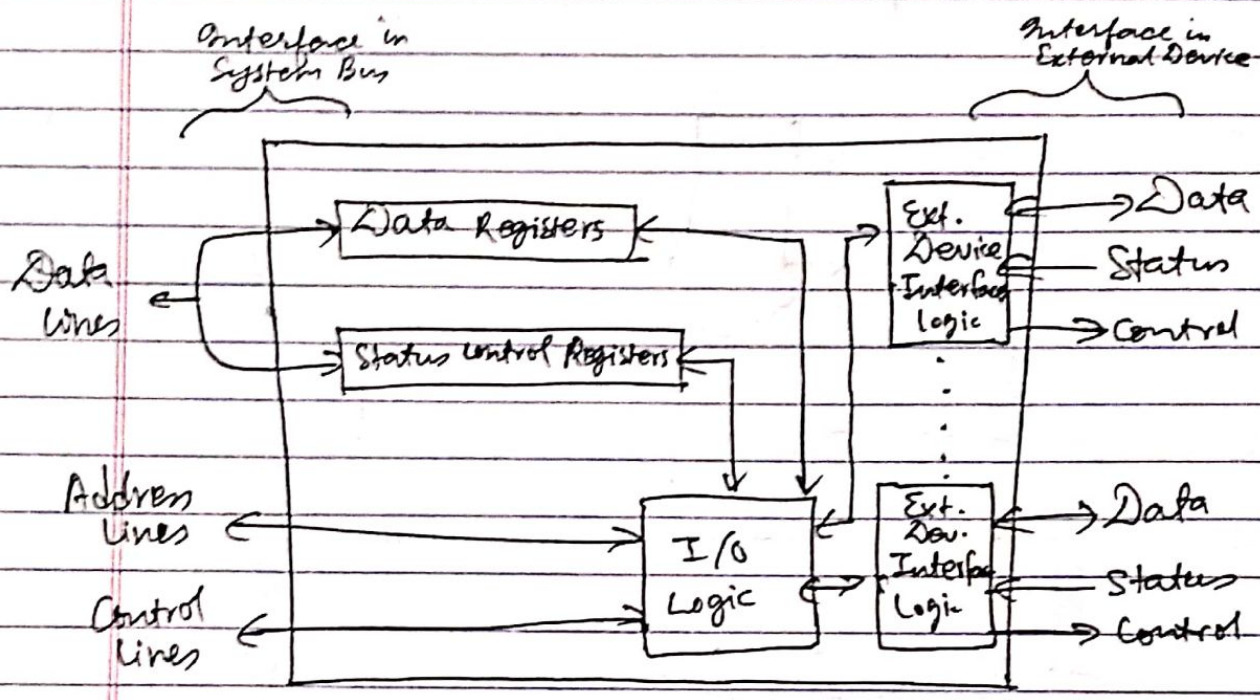


RAID 3 → high performance & uses parity for improved fault tolerance.



- RAID 4 : Similar to RAID 3, RAID 4 stripes data for high performance and uses parity for improved fault tolerance.

3. Block Diagram of I/O module. With each blocks.



General functions of IO modules can be listed as:

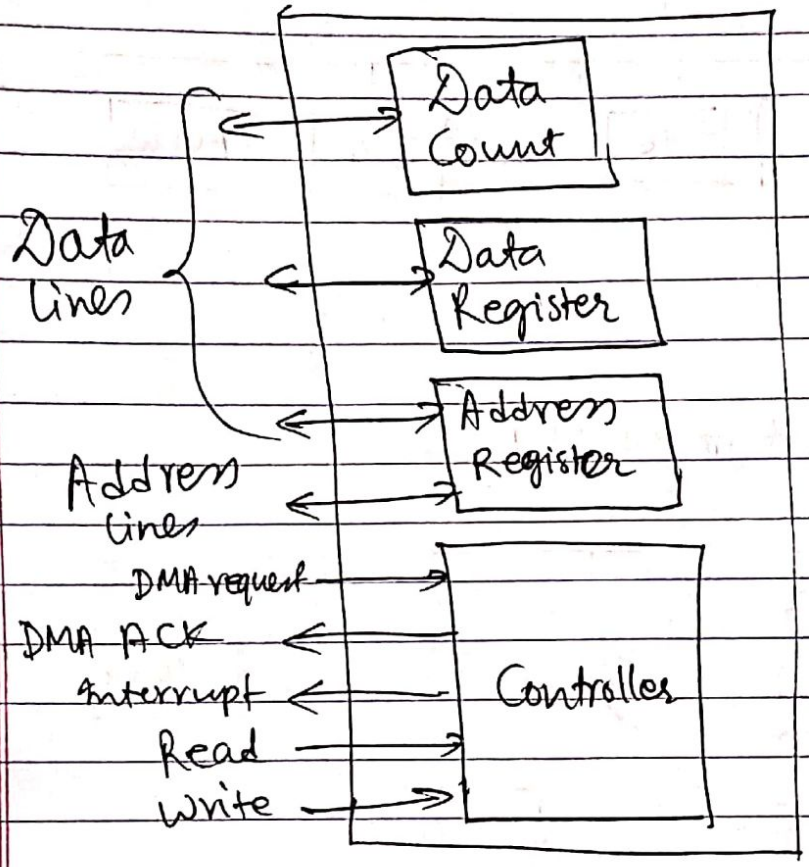
1. Processor communication
2. Device communication
3. Control & timing
4. Data Buffering

I/O Module Decisions

- Hide or reveal device properties to CPU
- Support multiple or single devices
- Control device functions or leave for CPU.
- Also O/S decisions.

e.g. Unix treats everything as file

4. DMA



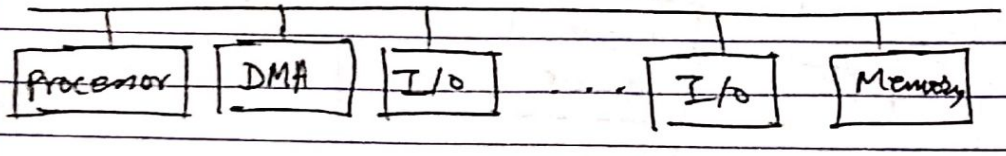
DMA

→ allows devices to transfer data without subjecting the processor a heavy overhead. Otherwise, the processor would have to copy each piece of data from the source to the destination.

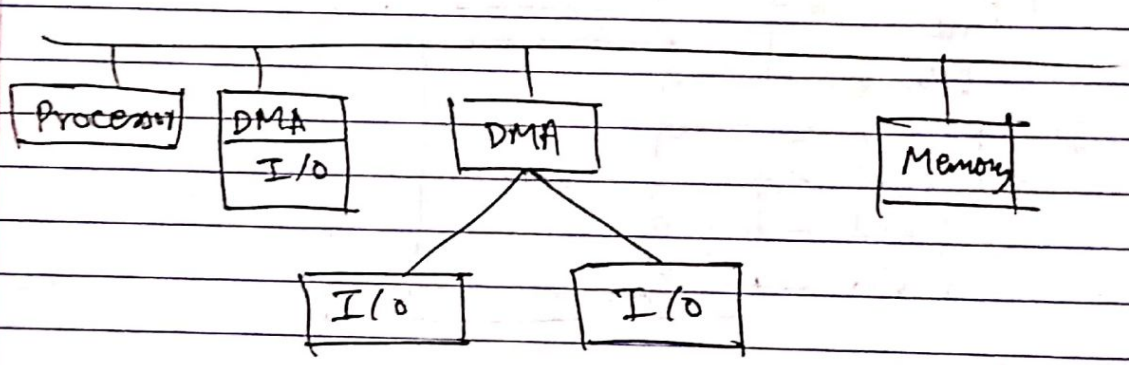
When large volumes of data is to be transferred, a more efficient way to do this is DMA.

Configurations of DMA:

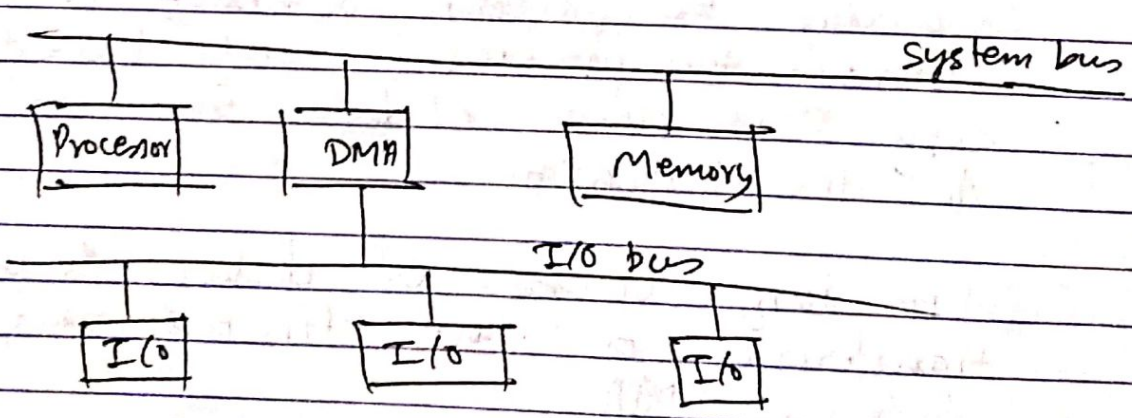
(a) Single bus, detached DMA:



(b) Single-bus, integrated DMA-I/O



(c)



CHAPTER 4

1. Explain use of pipelining.

→ In computers, a pipelining is the continuous and somewhat overlapped movement of instruction to the processor or in the arithmetic steps taken by the processor to perform an instruction.

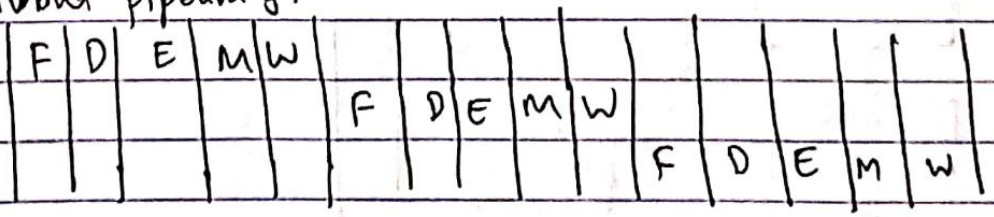
We can improve CPU performance in two ways.

(1) Improve the hardware by introducing faster circuits.
(2) Arrange the hardware such that more than one operation can be performed at the same time.

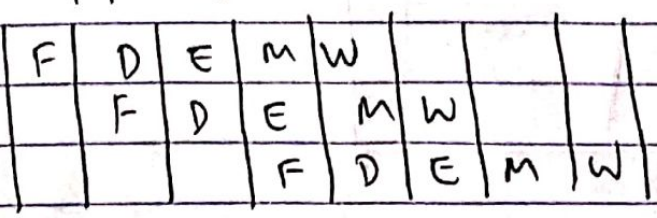
Pipelining is a process of arrangements of hardware elements of the CPU such that its overall performance is increased. Simultaneous execution of more than one instruction takes place in a pipelined processor.

If F = Fetch instruction, D = Decode, E = Execute, M = Store in memory, W = Write Back,

Without pipelining:



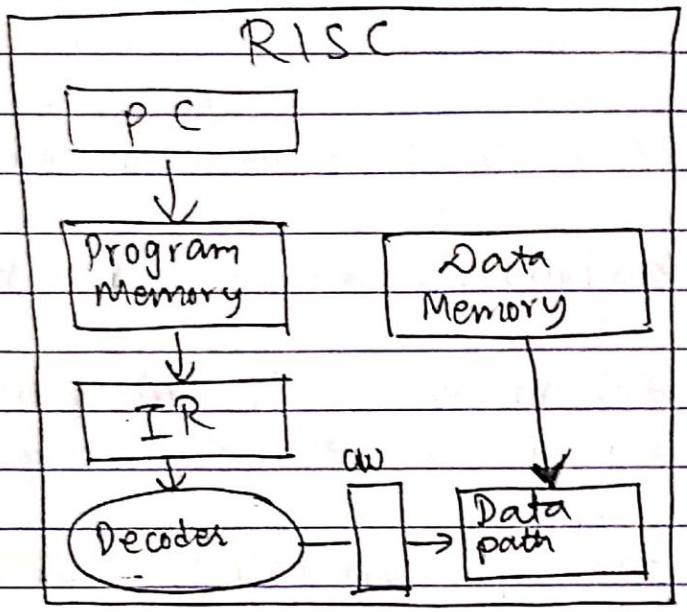
With pipelining:



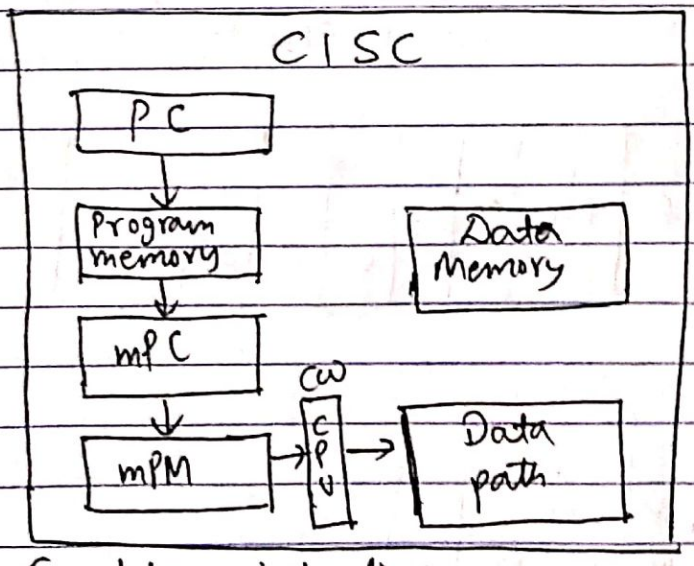
With pipelining, more number of instructions ~~can be used of~~ cycle can happen in less time thereby increasing the performance.

3. RISC and CISC Differentiation

→ CPU design plan based on simple orders and acts fast.



Simple instructions [one instruction = One CW ~~RAM~~ PM longer.]



Complex instructions

[One instruction = several CW PM standard]

4. Types of dependencies and conflicts in the instruction level parallelism with suitable examples.

(a) Data Dependencies
(b) Name dependencies
(c) Control dependencies.

(a) Data Dependency: When an instruction attempts to use resource before it is ready. As a result, it leads to inconsistency. The problem if not solved, the calculation is performed on old value instead of the updated value.

e.g. Add R_1, R_2, R_3
Sub R_5, R_1, R_6

Lets see this in Data path Diagram:

Inst ↓ CLK →	1	2	3	4	5	6
Add	IF	ID (R_2+R_3)	E	M (R_1)	W	
Sub		IF	ID (R_1-R_2)	E	M	W

whatever instruction is ~~fetched in Sub~~ ~~st~~ second step, its not the updated value of R_1. So there is error.

Data Dependency error are of three types:
RAW → read after write
WAR → write after write
WAW ⇒ Write after write.

Name Dependency occurs when two instructions use the same register or memory location, but there is no flow in data between the instructions associated with that name.

© Control Dependency.

Control Dependency are most dangerous dependency because of their branching natures& transfer of control.
Dependency of instructions to the sequential flow of execution and preserves branch (or any flow altering operation) behavior of the program.

So to solve this problem, whatever is fetches until now should be stalled.

Explain normal and delayed branch in RISC machine with suitable codes and diagrams. Differentiate between cache and registers.

→ One way to maximize the use of the pipeline, is to find an instruction that can be safely executed whether the branch is taken or not, and execute that instruction.

When the program was compiled, the compiler looked at each branch instruction, and tried to find something that could be safely executed, whether we take the branch or not.

Here,

```
LOOP: LW   R1, 0(R2)
      SUB  R3, R3, R1
      BNEZ R3 LOOP
      SW   R4, 0(R3)
```

we couldn't execute the SW R4, 0(R3) each time through the loop without causing any problems, because the value of R3 is changing in our loop. But, suppose instead, we had the instruction each time through the loop with no problems. We would simply be writing repeatedly to the same memory location, and once we finally do exist the loop, we will just continue on normally, no harm done. However, we are also still wasting a cycle for each time we DO go through the loop.

A better strategy is for the compiler to find an instruction either from inside the loop, or from before the loop, that can safely be rescheduled.

For e.g.

```
LOOP: SUB  R3, R3, R1
      SW   R3, 0(R5)
      BNEZ R3 LOOP
      SW   R4, 0(R6)
```